```
#Anshul Hudda
#cu24250002
#labsheet 9
```

```
import torch

print("PyTorch version:", torch.__version__)
print("GPU Available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU Name:", torch.cuda.get_device_name(0))
else:
    print("Running on CPU")
```

```
PyTorch version: 2.11.0+cu128
GPU Available: True
GPU Name: Tesla T4
```

```
!pip install -q ultralytics
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.5/46.5 kB 2.1 MB/s et
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.4/1.4 MB 35.5 MB/s eta 0
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 78.4/78.4 kB 8.4 MB/s eta
```

```
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
import numpy as np

print("All libraries imported successfully!")
```

```
Creating new Ultralytics Settings v0.0.8 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ul
Update Settings with 'yolo settings key=value', i.e. 'yolo settings ru
All libraries imported successfully!
```

```
# Load the pre-trained YOLOv8 model
model = YOLO("yolov8n.pt")

print("YOLOv8 model loaded successfully!")
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8
YOLOv8 model loaded successfully!
```

```
# Download a sample test image
!wget -q https://ultralytics.com/images/bus.jpg -O bus.jpg

print("Image downloaded successfully!")
```

```
Image downloaded successfully!
```

```
# Read and display the input image
image = cv2.imread("bus.jpg")

# Convert BGR to RGB for displaying with Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 7))
plt.imshow(image_rgb)
plt.axis("off")
plt.title("Input Image")
plt.show()
```



```
import time
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

image_path = "bus.jpg"

image = cv2.imread(image_path)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 7))
plt.imshow(image_rgb)
plt.axis("off")
plt.title("Input Image")
plt.show()

start_time = time.time()
results = model(image_path, conf=0.5)
end_time = time.time()

inference_time = end_time - start_time

print("Inference Time:", round(inference_time, 4), "seconds")
print("Approximate FPS:", round(1 / inference_time, 2))

result = results[0]

detected_image = result.plot()

plt.figure(figsize=(12, 8))
plt.imshow(cv2.cvtColor(detected_image, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("YOLOv8 Object Detection")
plt.show()

print("\nDetected Objects")
print("=" * 40)

for i, box in enumerate(result.boxes, start=1):
    class_id = int(box.cls[0])
    class_name = model.names[class_id]
    confidence = float(box.conf[0])
    coordinates = box.xyxy[0].tolist()

    print(f"\nObject {i}")
    print("Class:", class_name)
    print("Confidence:", round(confidence, 3))
    print("Bounding Box:", [round(x, 2) for x in coordinates])

object_counts = {}

for box in result.boxes:
    class_id = int(box.cls[0])
```

```
        class_name = model.names[class_id]
        object_counts[class_name] = object_counts.get(class_name, 0) + 1

    print("\nObject Count")
    print("=" * 40)

    for name, count in object_counts.items():
        print(f"{name}: {count}")

    print("\nObject detection completed successfully!")
```




```
image 1/1 /content/bus.jpg: 640x480 3 persons, 1 bus, 6.5ms
Speed: 105.5ms preprocess, 6.5ms inference, 42.4ms postprocess per ima
Inference Time: 8.1326 seconds
Approximate FPS: 0.12
```




```
# Experiment No. 9
# Object Detection and Recognition using Deep Learning Models on Stan

# Q1. Differentiate between image classification, object detection, a
# Image Classification assigns class labels to an entire image.
# Object Detection identifies objects and locates them using bounding
# Image Segmentation classifies individual pixels to determine the ex
#
# Q2. Explain the working principle of YOLO.
# YOLO stands for You Only Look Once.
```

```
# It processes the complete image in a single forward pass through a
# It predicts bounding boxes, class labels, and confidence scores.
# Low-confidence detections can be removed using a confidence thresho
# Non-Maximum Suppression (NMS) is used to remove duplicate overlappi
# YOLO is fast and is suitable for real-time object detection.
#
# Q3. Compare YOLO, SSD, and Faster R-CNN.
# YOLO is a one-stage detector with very high detection speed.
# SSD is also a one-stage detector and provides a good balance betwee
# Faster R-CNN is a two-stage detector and generally provides strong
# but requires more computation and is usually slower than YOLO and S
# YOLO and SSD are commonly used for real-time applications, while Fa
# is often used when detection quality is prioritized.
#
# Q4. What is a bounding box? Why is it important?
# A bounding box is a rectangular box surrounding a detected object.
# It represents the position and size of the object in an image.
# It is important because it allows the system to locate and distingu
#
# Q5. Explain confidence score and IoU.
# A confidence score indicates how confident the model is about a par
# Higher confidence generally indicates greater confidence in the pre
# IoU stands for Intersection over Union.
# IoU measures the overlap between a predicted bounding box and the g
# IoU = Area of Intersection / Area of Union.
# IoU is used to evaluate the quality of object localization.
#
# Q6. Advantages of pre-trained deep learning models.
# Pre-trained models have already learned useful features from large
# They reduce the need to train a model from scratch.
# They can recognize complex patterns and objects.
# They require less manual feature engineering.
# They can be fine-tuned for specific applications.
# They are generally more robust than traditional rule-based image-pr
#
# Q7. Why are MS COCO and Pascal VOC widely used?
# MS COCO and Pascal VOC contain large collections of real-world imag
# They provide annotated objects and bounding boxes.
# They contain multiple object categories.
# They provide standard training and testing datasets.
# They have commonly used evaluation metrics.
# They are useful for comparing different object detection algorithms
#
# Q8. Five real-world applications of object detection.
# 1. Autonomous vehicles - detecting cars, pedestrians, and traffic s
# 2. Surveillance systems - detecting people and vehicles.
# 3. Retail analytics - detecting products and monitoring shelves.
# 4. Healthcare - detecting abnormalities in medical images.
# 5. Industrial inspection - detecting defects and components.
#
# Q9. Challenges in complex real-world object detection.
# Common challenges include:
# 1. Occlusion of objects.
# 2. Poor lighting conditions.
# 3. Very small objects.
```

```
# 4. Different object sizes and scales.
# 5. Complex or cluttered backgrounds.
```